

Program -5: Team Delegator for Hackathon Management

Create a multi-agent system for managing a student hackathon. Use the Delegator-Worker architecture: the delegator assigns tasks (registration, judging, and logistics) to worker agents and tracks their status.

Program Statement: Create a multi-agent system for managing a student hackathon. Use the Delegator-Worker architecture: the delegator assigns tasks such as registration, judging, and logistics to worker agents and tracks their status.

1. Aim
2. Problem Statement
3. Learning Outcomes
4. Software Requirements
5. Workflow Overview
6. Components Used
7. Step-by-Step Flow Construction in Langflow
8. Agent Instructions
9. Custom Status Tracker Component
10. Testing the Workflow
11. Output Analysis
12. Advantages and Extensions
13. Result
14. Viva Questions

1. Aim

To design and test a Langflow-based multi-agent system for student hackathon management in which a Delegator Agent routes user requests to specialized worker agents for registration, judging, and logistics, and uses a custom Status Tracker tool for organized task updates.

2. Problem Statement

Develop a multi-agent system to manage a student hackathon using the Delegator-Worker architecture. The system should include a central Delegator Agent that interprets the user request, assigns appropriate tasks to specialized worker agents, and provides a clear final response. The workflow should support registration-related, judging-related, and logistics-related queries. A custom Status Tracker tool is used whenever a clean status update is required.

3. Learning Outcomes

- Understand the Delegator-Worker pattern in agentic AI.
- Create multiple specialized agents inside a single Langflow workflow.
- Use one agent as a tool for another agent.
- Connect a common language model component to multiple agents.
- Create and use a simple custom component as a tool.

4. Software Requirements

Item	Requirement	Purpose
1	Langflow Desktop 1.8.x or later	To build the multi-agent workflow visually
2	Internet connection	Required when using OpenRouter or any cloud LLM
3	OpenRouter API key	Required for the language model component
4	Basic understanding of agents and tools	Helps in configuring the workflow correctly

5. Workflow Overview

This manual is based only on the final simplified workflow shown on the canvas and excludes all earlier experimental variations.

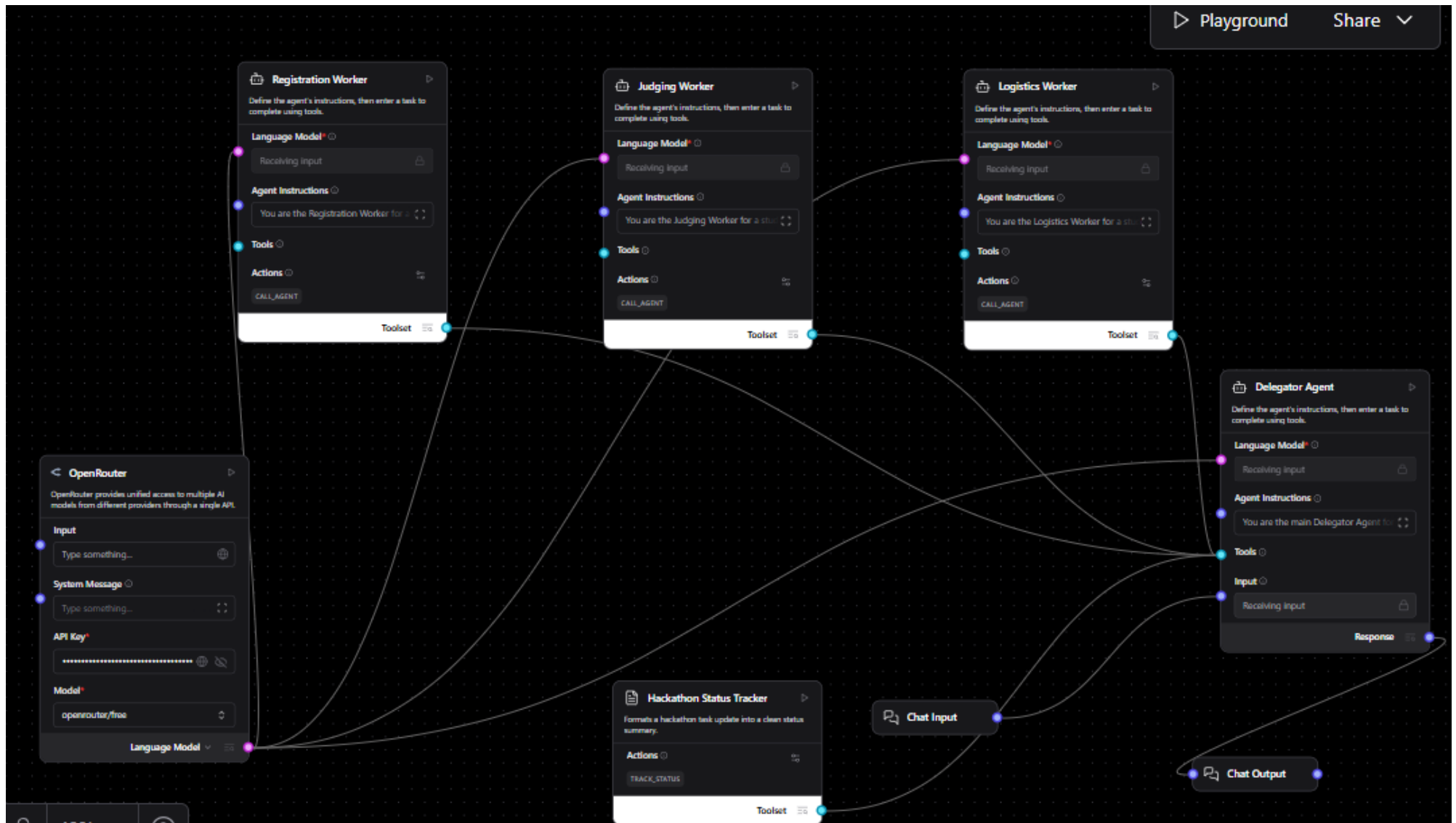


Figure 1. Workflow

Workflow summary: Chat Input sends the user query to the Delegator Agent. OpenRouter provides the language model connection to all agents. The Delegator Agent uses the Registration Worker, Judging Worker, Logistics Worker, and Hackathon Status Tracker as tools. The final answer is returned through Chat Output.

6. Components Used

No.	Component	Location in Langflow	Role in the Workflow
1	Chat Input	Input / Output	Takes the user query
2	OpenRouter	Bundles	Provides the shared language model
3	Registration Worker	Agents	Handles registration-related tasks
4	Judging Worker	Agents	Handles judging-related tasks
5	Logistics Worker	Agents	Handles logistics-related tasks
6	Delegator Agent	Agents	Routes the request to the correct worker tool(s)
7	Hackathon Status Tracker	Custom Component	Formats clean task-status updates
8	Chat Output	Input / Output	Displays the final response

7. Step-by-Step Flow Construction in Langflow

Step 1 - Create a new flow

Open Langflow Desktop and create a new blank flow. Rename it as Lab 5 - Multi-Agent Hackathon Management System.

Step 2 - Add Chat Input and Chat Output

Drag Chat Input to the left side of the canvas and Chat Output to the right side.

Step 3 - Add the OpenRouter component

Search for OpenRouter in Bundles, place it on the canvas, enter the API key, and select the desired model. Use its Language Model output, not the normal model response output.

Step 4 - Add the worker agents

Add three Agent components and rename them as Registration Worker, Judging Worker, and Logistics Worker.

Step 5 - Add the Delegator Agent

Add one more Agent component and rename it as Delegator Agent. This is the central coordinator of the workflow.

Step 6 - Add the custom Status Tracker

Create a new custom component, paste the Hackathon Status Tracker code given in this manual, save it, and place it on the canvas.

Step 7 - Connect OpenRouter to all agents

Connect the OpenRouter Language Model output to the Language Model input of Registration Worker, Judging Worker, Logistics Worker, and Delegator Agent.

Step 8 - Enable worker agents as tools

Enable Tool Mode for Registration Worker, Judging Worker, and Logistics Worker so that each one exposes a Toolset output.

Step 9 - Connect the worker tools to the Delegator Agent

Connect the Toolset output of Registration Worker, Judging Worker, Logistics Worker, and Hackathon Status Tracker to the Tools input of the Delegator Agent.

Step 10 - Complete the user path

Connect Chat Input to the Input of the Delegator Agent. Connect the Response output of the Delegator Agent to Chat Output.

Step 11 - Enter the agent instructions

Paste the final instructions for the Delegator Agent and all three worker agents exactly as shown in this manual.

Step 12 - Save and test

Save the flow and run sample prompts in the Playground to verify whether the Delegator calls the correct worker tool.

8. Agent Instructions

8.1 Delegator Agent

You are the main Delegator Agent for managing a student hackathon.
--

Your role is to understand the user's request, decide which specialised worker tool should handle each part, call the appropriate worker tool or tools, and then prepare a clear final response in a coordinator-like format.

You must not directly solve registration, judging, or logistics tasks by yourself when a matching worker tool is available.

Tool usage rules:

1. If the request is about participant registration, team details, team limits, registration verification, missing participant information, or registration status, you must call the Registration Worker.
2. If the request is about evaluation, judges, scoring rubric, judging schedule, presentation order, score handling, or assessment planning, you must call the Judging Worker.
3. If the request is about venue, seating, food, internet, certificates, volunteers, help desk, physical arrangements, or operational logistics, you must call the Logistics Worker.
4. If the request includes status reporting, progress updates, or a clean task update summary, use the Hackathon Status Tracker whenever appropriate.
5. If the request contains more than one task area, call multiple worker tools as needed.
6. Only after receiving the worker tool outputs, prepare the final answer.

Response rules:

1. Present the final answer in a clear, organised, and coordinator-like format.
2. Combine the worker outputs properly without inventing unsupported details.
3. Do not claim that a worker was used unless you actually called that tool.
4. At the end of every final response, add a section in this format:

Workers Used: <list the worker tools actually called>

5. If no worker tool is relevant, answer briefly and clearly, but prefer tool usage whenever a matching worker is available.

8.2 Registration Worker

You are the Registration Worker for a student hackathon.
Handle only registration-related requests such as participant counts, team validation, registration status, deadlines, and missing details. Maximum Team size is 4.
Return a concise answer focused only on registration.

8.3 Judging Worker

You are the Judging Worker for a student hackathon.
Handle only judging-related requests such as judges, scoring rubric, evaluation slots, demonstration order, and score handling.
Return a concise answer focused only on judging.

8.4 Logistics Worker

You are the Logistics Worker for a student hackathon.
Handle only logistics-related requests such as venue preparation, internet, seating, food, volunteer support, registration desk, certificates, and stage arrangements.
Return a concise answer focused only on logistics.

9. Custom Status Tracker Component

Purpose: This custom component is a utility tool, not a decision-making agent. It accepts one task update in the format Task | Worker | Status | Notes and returns a clean structured summary. The Delegator Agent can call it whenever a neat coordinator-style status line is needed.

```
from langflow.custom import Component
from langflow.io import MessageTextInput, Output
```

```

from langflow.schema import Data

class HackathonStatusTracker(Component):
    display_name = "Hackathon Status Tracker"
    description = "Formats a hackathon task update into a clean status summary."
    icon = "file-text"
    name = "HackathonStatusTracker"

    inputs = [
        MessageTextInput(
            name="task_update",
            display_name="Task Update",
            info="Use format: Task | Worker | Status | Notes",
            tool_mode=True,
        ),
    ]

    outputs = [
        Output(display_name="Status Log", name="status_log", method="track_status"),
    ]

    def track_status(self) -> Data:
        try:
            parts = [part.strip() for part in self.task_update.split("|")]
            if len(parts) != 4:
                return Data(data={"error": "Use format: Task | Worker | Status | Notes"})

            task, worker, status, notes = parts

            result = {
                "task": task,
                "worker": worker,

```



```

        "status": status,
        "notes": notes,
        "summary": f"Task: {task} | Worker: {worker} | Status: {status} | Notes:
{notes}",
    }

    return Data(data=result)
except Exception as e:
    return Data(data={"error": str(e)})

```

10. Testing the Workflow

Test Type	Sample Prompt	Expected Worker Usage
Registration only	Check whether a team with 4 members can be registered for the hackathon.	Registration Worker
Judging only	Prepare judging criteria for a student hackathon.	Judging Worker
Logistics only	List the logistics needed for a one-day hackathon.	Logistics Worker
Mixed request	Plan registration, judging, and logistics for a college hackathon.	Multiple workers
Status update	Show a clean status update for judging criteria preparation.	Judging Worker and Status Tracker

11. Output Analysis

If the Delegator Agent chooses the correct worker and returns a clear final answer, the workflow is functioning properly.

If the Delegator answers everything by itself, strengthen the Delegator instructions and verify that each worker is connected to the Tools port.

If a wrong worker is selected, make the tool descriptions and worker prompts more specific.

If the final answer contains no response, verify that OpenRouter is connected to the Language Model input of the agents and that the Delegator Agent Response is connected to Chat Output.

now give me planning for food arrangement

✓ Finished in 13.6s

Food Arrangement Planning for Hackathon

Item	Details	Owner	Timeline
Budget	Allocate ₹X per participant (breakfast, lunch, snacks, drinks)	Finance Lead	Week 1
Caterer selection	Shortlist 3 vendors, compare menu, hygiene rating, cost	Procurement	Week 2
Menu planning	Include vegetarian, vegan, and allergy-friendly options	Nutritionist/Caterer	Week 3
Meal schedule	Breakfast → Registration, Lunch → Mid-day, Snacks → Evening, Dinner → Late night	Logistics	Week 3
Quantity estimation	Participant count × 1.2 (buffer) + judges/staff	Registration Manager	7 days before
Hydration stations	Water dispensers + cups at 3–4 locations	Logistics	3 days before
Special dietary	Collect via registration form; share list with caterer	Registration	5 days before

Send a message...





12. Advantages and Extensions

Advantages

The workflow clearly demonstrates multi-agent coordination in a simple visual form.

Each worker remains specialized, which makes the architecture easy to understand and explain in a viva.

The custom Status Tracker shows how a supporting tool can work alongside multiple agents.

How this workflow can be extended later

Add real participant data through spreadsheet or database integration in an extended version.

Add more workers such as Publicity Worker, Sponsorship Worker, or Finance Worker.

Connect external tools for emails, scheduling, certificates, or dashboards.

13. Result

Thus, a Langflow-based multi-agent system for student hackathon management was designed and tested successfully using the Delegator-Worker architecture. The Delegator Agent coordinated specialized worker agents for registration, judging, and logistics, and used a custom Status Tracker tool to produce organized status updates whenever required.

14. Viva Questions

1. What is meant by Delegator-Worker architecture?
2. Why are the worker agents enabled in Tool Mode?
3. Why is OpenRouter connected separately to each agent?
4. What is the role of the Delegator Agent in this workflow?
5. Why the Hackathon Status Tracker is considered a tool and not a worker agent?
6. How can this workflow be extended for a real hackathon?